

Funksjoner

Kapittel 3

Fra «*en liste med kommandoer*» til «*noe som er bygd av deler*»

Notatbok: 3 - Funksjoner.ipynb

Plan for forelesningen

Det vi skal igjennom

3.1 Innebygde funksjoner i Python ↗

3.2 Hva er egentlig en funksjon?

3.3 Å definere egne funksjoner (def) syntaksen

3.4 Parametere vs. argumenter

3.5 return vs. print

dagens viktigste slide

3.6 Korte, anonyme funksjoner (lambda)

3.7 Caseoppgave: funksjoner og AI

Etter forelesningen skal du kunne

1. Se når kode bør pakkes inn i en funksjon.
2. Skrive en funksjon med `def`, parametere og `return`.
3. Forklare forskjellen på en *parameter* og et *argument*.
4. Forklare hvorfor `print` inne i en funksjon ofte er en feil.
5. Bruke en funksjon til å regne på renter, skatt, tilbud og etterspørsel.

Lister, tupler og NumPy kommer i kapittel 4. I dag holder vi oss til funksjoner.

Der vi slapp: kapittel 2

Så langt har vi lært å gi datamaskinen **kommando-**er:

- lagre en verdi i en variabel
- velge riktig datatype (int, float, str, bool)
- skrive ut med print() og f-strenger

Et program på den formen er en *liste* som kjøres ovenfra og ned.

X = 10 variabel
- y = 5 tilordning

Hva endrer seg nå?

Vi slutter å skrive lister, og begynner å bygge **deler**.

Tre nye ting:

1. **Navn på en kodeblokk** (så du kan bruke den igjen)
2. **Verdier inn** (parametere)
3. **Verdi ut** (return)

Alt fra kapittel 1 og 2 — variabler, datatyper, f-strenger — gjelder **fortsatt**.

Spørsmål fra sist?

Linjere under mult. x og y

x = 10

y = 5

multiplikasjon = x * y # din kommentar

! Alt som er bak #
følges ikke som
Python kode.

Spørsmål fra sist?

$$100 \cdot (1 + 0.10)$$

↑

↑

100%

10%

eldt rente

$$0 - 100\% \rightarrow 0 - 1$$

$$50\% \rightarrow 0.50$$

$$300.000,-$$

4% årlig rente

$$300.000 \cdot (1 + 0.04)^n$$

3.1

Hvorfor i det hele tatt funksjoner?

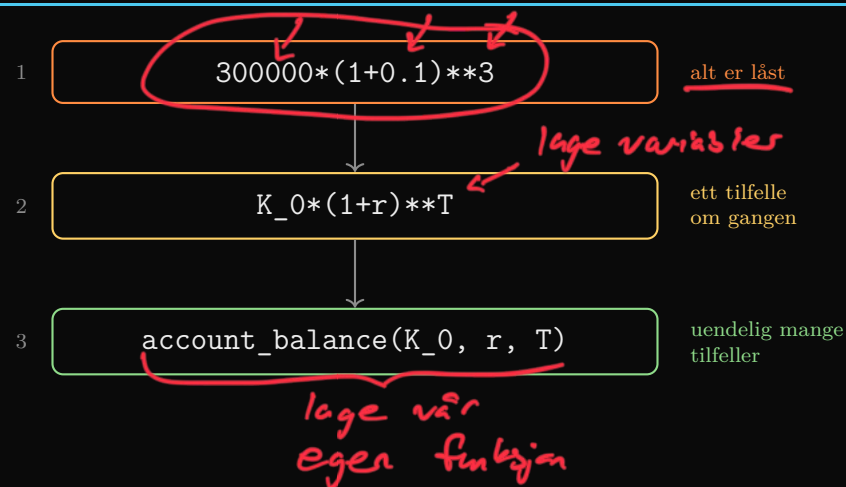
Problemet: koden vokser, og den gjentar seg

En kunde har lån på 300 000 kr, renta er 10 %, og vi vil vite beløpet om 3 år.

$300000 * (1 + 0.1) ** 3$
 $40000 * (1 + 0.1) ** 3$ } 50 ganger daglig

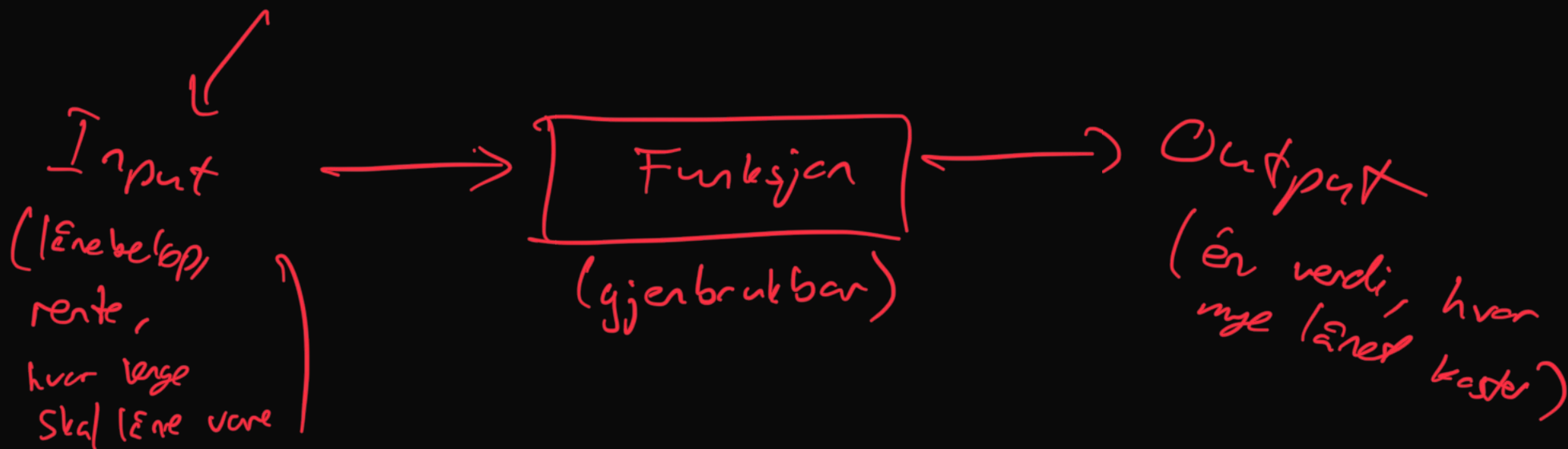
Banken ringer: renta er ikke 10 %, men 4,5 %.

Hvor mange steder må du rette? Og hva hvis du hadde 200 kunder?

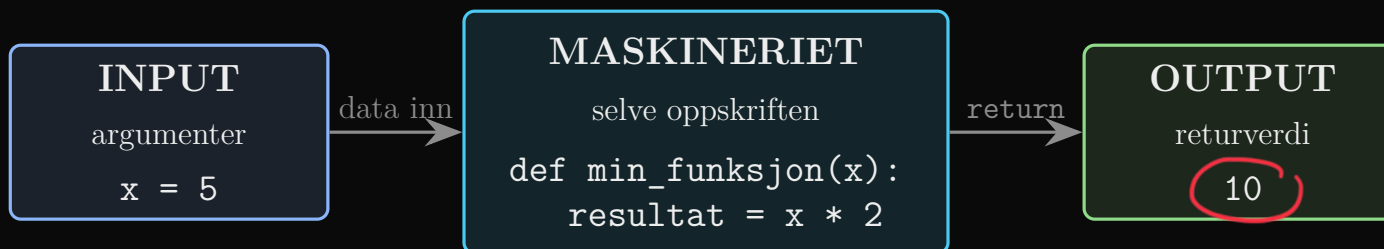


Hovedregelen i kapittel 3

Har du kode som gjentas, skal du lage en funksjon som håndterer den.



Hva er en funksjon? Tenk fabrikk



Definisjon:

En funksjon er en **gjenbrukbar kodeblokk** som tar inn noen verdier, gjør en beregning, og sender et resultat tilbake.

Når maskinen først er bygd, kan du mate den med nye tall så mange ganger du vil.

Hvorfor bygge slike maskiner?

- **Gjenbruk:** skriv én gang, bruk overalt
- **Ryddighet:** koden blir modulær og lesbar
- **Feilsøking:** én feil rettes ett sted
- **Samarbeid:** andre kan bruke maskinen din uten å se inn i den

Tegn din egen fabrikk: hva går inn, hva kommer ut?

Innebygde funksjoner — vi har brukt dem hele tiden

```
print(max(5, 3.0, 50))    # 50
print(min(2.4, 2.0))      # 2.0
print(abs(-2.4))          # 2.4
print(len('156'))         # 3
print(sum([1, 2, 3]))     # 6
print(type(3.14))         # <class 'float'>
```

Tenk før du svarer

Hva gir `len('156')`?

Ikke 156. Svaret er **3** — `len` teller *tegn*, ikke verdi. Anførselstegnene gjør 156 om til tekst.

RETURNERER noe

`max min abs len sum`

svaret sendes tilbake og kan brukes videre

GJØR noe

`print`

skriver på skjermen, sender ingenting tilbake

Legg merke til at `max`, `min`, `abs` og `len` står *inni* `print`. Det er nettopp fordi de returnerer et svar som noen andre må ta imot.

Notater

`int(14.5)` → 14

Låne andres funksjoner: `import numpy as np`

```
import numpy as np
```

```
np.exp(1)      # 2.718281828459045
```

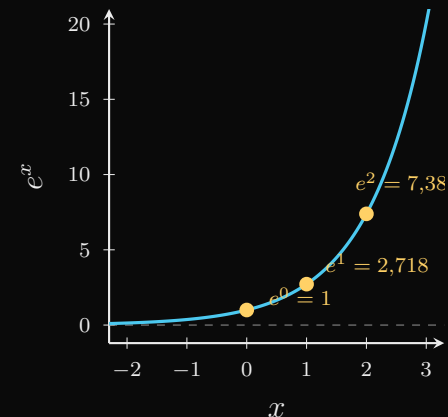
```
np.exp(2)      # 7.38905609893065
```

```
np.exp(1)**5    # 148.41315910257657
```

```
np.exp(5)      # 148.4131591025766
```

Eulers tall $e \approx 2,71828$. Eksponentialfunksjonen e^x finnes ikke innebygd i Python, men ligger i pakken `numpy`.

Se på de to siste linjene: matematisk er de identiske, men datamaskinen gir ulikt siste siffer. Flyttall har endelig presisjon — derfor sammenligner vi aldri desimaltall med `==`.



Logaritmen `np.log` er den motsatte funksjonen:

$$\log(e^x) = x \text{ og } e^{\log(x)} = x.$$

Notater

3.3

Å definere egne funksjoner

Funksjonens anatomi

def account_balance(K_0, r, T):

nøkkelord
«definer»

navnet du velger

akkurat det
du vil :)

KOLON
glemmes alltid

PARAMETERE
(tomme bokser)

disse skal brukes
← "inn" funksjon.

Sanne Jan
i notebook

```
def account_balance(K_0, r, T):  
    return round(K_0*(1+r)**T, 2)  
account_balance(300000, 0.1, 3) # 399300.0
```

innrykk

kalle på funksjonen

Seks ting, i denne rekkefølgen

1. def
2. navn
3. (parametere)
4. kolon :
5. innrykk (4 mellomrom)
6. return

Skriv funksjonen din her

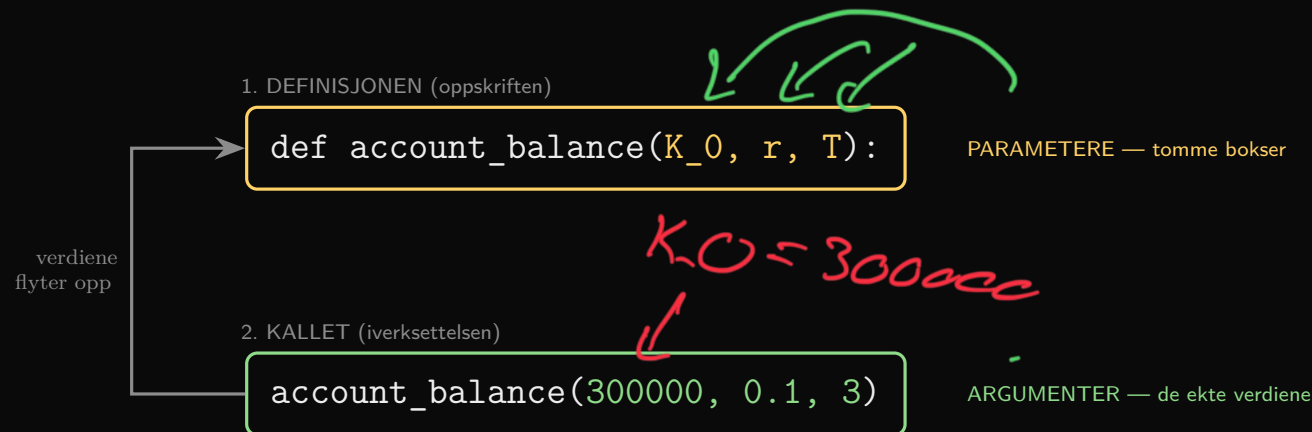
De tre feilmeldingene du kommer til å få

Feilmelding	Hva du gjorde	Fiks
 SyntaxError	Glemt kolon etter def ...)	Sett inn :
 IndentationError	Glemt innrykk, eller blandet tabulator og mellomrom	4 mellomrom, konsekvent
 NameError: name 'x' is not defined	Kalt funksjonen før cellen som definerer den ble kjørt	Restart Kernel and Run All Cells

Feilmeldinger er ikke nederlag — de er beskjeder. Les den *siste* linja først: der står typen feil og hva Python forventet. Linjenummeret over forteller hvor.

Notater fra tavla

Parameter vs. argument



Parameteren er merkelappen på esken.
Argumentet er det du faktisk legger i esken den dagen du sender den.

Sjekk at du har det

```
def kvadrat(x):          # x er PARAMETER
    return x**2

print(kvadrat(5))        # 5 er ARGUMENT -> 25
```

rekkefølge på argument.

Rekkefølgen betyr alt

```
account_balance(300000, 0.1, 3)
```

```
# 399300.0
```

```
account_balance(0.1, 300000, 3)
```

```
# 270000270000090000.0
```

Ingen feilmelding

Python protesterte ikke med ett ord. Den regnet lydig ut

$$0,1 \cdot (1 + 300\,000)^3.$$

Datamaskinen kan ikke se at du mente noe annet.

Redningen: navngitte argumenter

```
account_balance(K_0=300000, r=0.1, T=3)
```

```
account_balance(r=0.1, T=3, K_0=300000)
```

```
# begge gir 399300.0
```

Skriver du navnet foran, spiller rekkefølgen ingen rolle — og den som leser koden din om et halvt år (altså du) ser med én gang hva 0.1 var.

Den farligste feiltypen

Feil som gir *feilmelding* finner du på ett minutt.

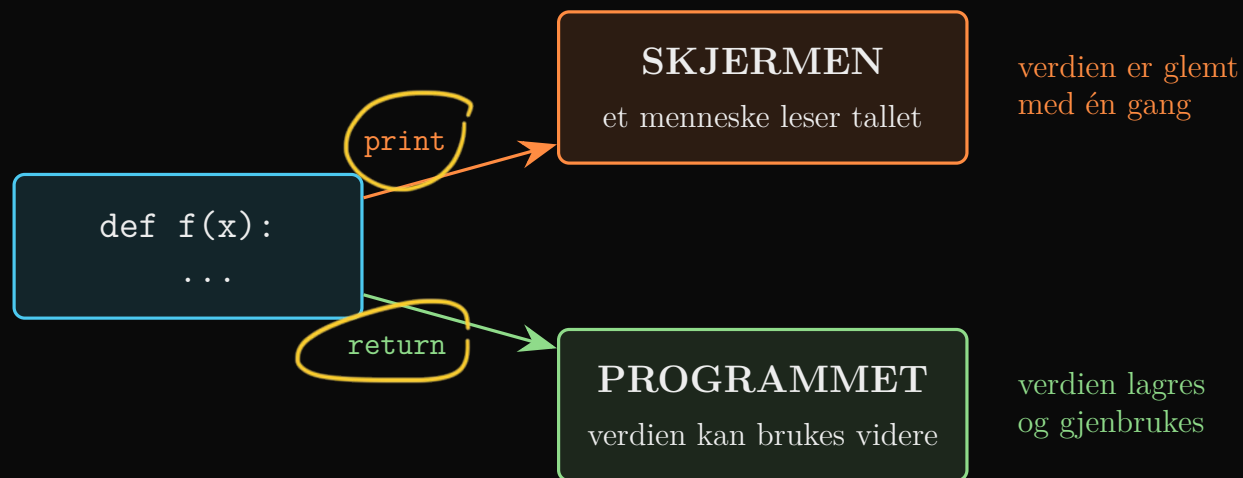
Feil som gir *sva*r kan du levere inn uten å merke det.

Notater

3.5

return eller print?

To helt forskjellige jobber



Gyllen huskeregel

Bruk `print()` når du skal snakke til et **menneske**.

Bruk `return` når funksjonen skal snakke videre til resten av **programmet**.

Har funksjonen ingen `return`, returnerer den automatisk `None` — «ingenting».

Notater

Og slik krasjer det

FEIL: funksjon uten return

```
def overskudd(inntekt, kostnad):  
    print(inntekt - kostnad)
```

```
mitt = overskudd(10000, 4000)  
mva = mitt * 0.25
```

6000

`TypeError: unsupported operand`

`type(s) for *: 'NoneType' and 'float'`

RIKTIG: funksjon med return

```
def overskudd(inntekt, kostnad):  
    return inntekt - kostnad
```

```
mitt = overskudd(10000, 4000)  
mva = mitt * 0.25  
print(f"MVA: {mva} kr")
```

MVA: 1500.0 kr

Den første funksjonen *printet* 6000. Det så helt riktig ut på skjermen. Men variabelen `mitt` ble stående tom, og ingenting ganger 0,25 er ikke 1500 — det er et krasj.

Notater

Kapittelcase 3: funksjoner og AI

Sjefen din ber ChatGPT om «en Python-funksjon som tar inn en pris, legger til 25 % skatt, og *viser* resultatet». Han får:

```
def legg_til_skatt(pris):  
    pris_med_skatt = pris * 1.25  
    print(pris_med_skatt)  
  
en_bok = legg_til_skatt(200)  
totalpris = en_bok * 3
```

```
TypeError: unsupported operand type(s)  
for *: 'NoneType' and 'int'
```

Din forklaring til sjefen

Forklar sjefen hva som skjedde

Ordet «viser» i bestillingen ble til `print`. Koden *kjører*, den ser riktig ut, og den er ubrukelig.

Dette er den vanligste feilen språkmodeller gjør med funksjoner.

Poenget er ikke å la være å bruke AI. Poenget er at du må kunne *lese* svaret godt nok til å fange dette — ellers leverer du kode som krasjer hos andre.

Anvendelse

Renteregning, skatt og markeder

Renteregning: fra formel til funksjon

Et lån på K_0 med rente r , forrentet én gang i året i T år:

$$K_T = K_0 (1 + r)^T$$

```
def account_balance(K_0, r, T):  
    return round(K_0*(1+r)**T, 2)
```

Legges renta til n ganger i året:

$$K_T = K_0 \left(1 + \frac{r}{n}\right)^{T \cdot n}$$

```
def account_balance_n(K_0, r, T, n):  
    return K_0*(1+r/n)**(T*n)
```

Regn selv: 100 kr, 20 %, 2 år

n	beløp
1 (årlig)	144,00
12 (månedlig)	148,69
365 (daglig)	149,16
31 536 000 (hvert sekund)	149,1825
∞ (kontinuerlig)	149,1825 = $100 e^{0,4}$

Sjekk 144 i hodet: $100 \cdot 1,2 \cdot 1,2$

Regn ut beløpet for 100 kr, 20 %, 2 år med $n = 4$

Standardverdier: argumenter som er valgfrie

```
def pris_med_mva(pris, sats=0.25):  
    return pris * (1 + sats)
```

```
pris_med_mva(1000)          # 1250.0  
pris_med_mva(1000, 0.15)    # 1150.0
```

Standardverdien er det funksjonen *antar* hvis du ikke sier noe. Nyttig når det finnes en vanlig verdi — for eksempel en mva-sats som nesten alltid er 25 %.

Samme navn to ganger: den siste vinner

```
def f(x):  
    return 2*x
```

```
f(8)          # 16
```

```
def f(x, a=102):  
    return a*x
```

```
f(8)          # 816
```

Derfor: Restart Kernel and Run All Cells før du leverer. Ellers kan koden gi ett svar hos deg og et annet hos meg.

Notater

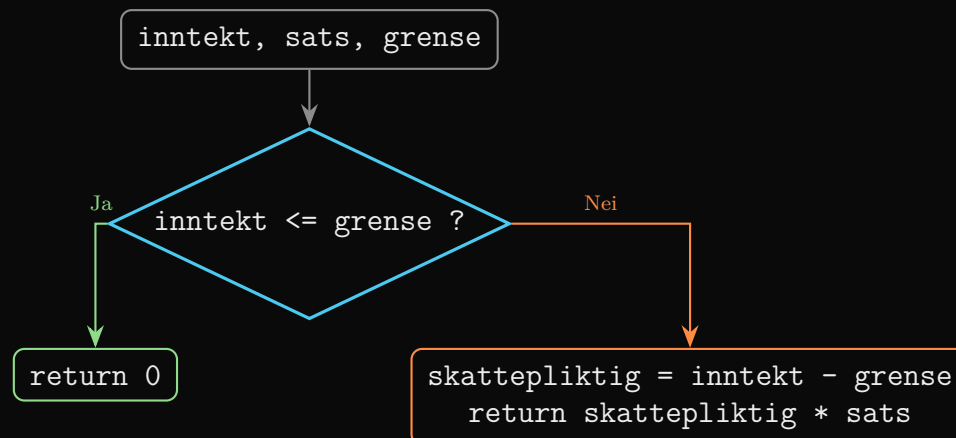
En funksjon kan velge: if inne i funksjonen

Pseudokode først — lukket laptop.

```
HVIS inntekt ≤ grense:  
  skatt = 0  
ELLERS:  
  skattepliktig = inntekt - grense  
  skatt = skattepliktig × sats  
RETURNER skatt
```

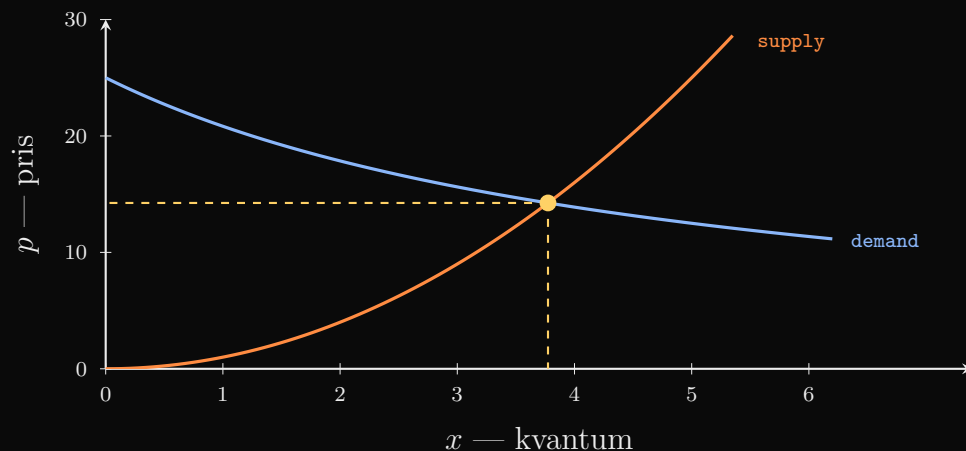
```
def skatt(inntekt, sats, grense):  
    if inntekt <= grense:  
        return 0  
    else:  
        return (inntekt - grense) * sats
```

```
skatt(100000, 0.25, 15000)    # 21250.0
```



Notater

Funksjoner som økonomiske modeller



```
def demand(x):  
    return 125/(5+x)
```

```
def supply(x):  
    return x**2
```

Merk retningen

Dette er **inverse** funksjoner: de tar inn *kvantum* og gir ut *pris*. I mikro har du som regel sett $x = f(p)$ — altså motsatt vei.

Oppgaver å jobbe med: finn med prøving og feiling hvilket kvantum som gir likevekt. Prøv $x = 3$: tilbud 9, etterspørsel 15,6 — for lavt. Prøv $x = 4$: tilbud 16, etterspørsel 13,9 — for høyt. Sirkle inn.

Prøving og feiling: skriv verdiene her

Korte, anonyme funksjoner: lambda

```
# Den tradisjonelle måten
def beregn_skatt(inntekt):
    return inntekt * 0.22

# Lambda-måten, alt på en linje
beregn_skatt_l = lambda inntekt: inntekt * 0.22

print(beregn_skatt(500000))      # 110000.0
print(beregn_skatt_l(500000))   # 110000.0
```

Syntaksen er alltid `lambda` parametere: uttrykk.
Legg merke til at det *ikke* står `return` — den er usynlig og alltid der.

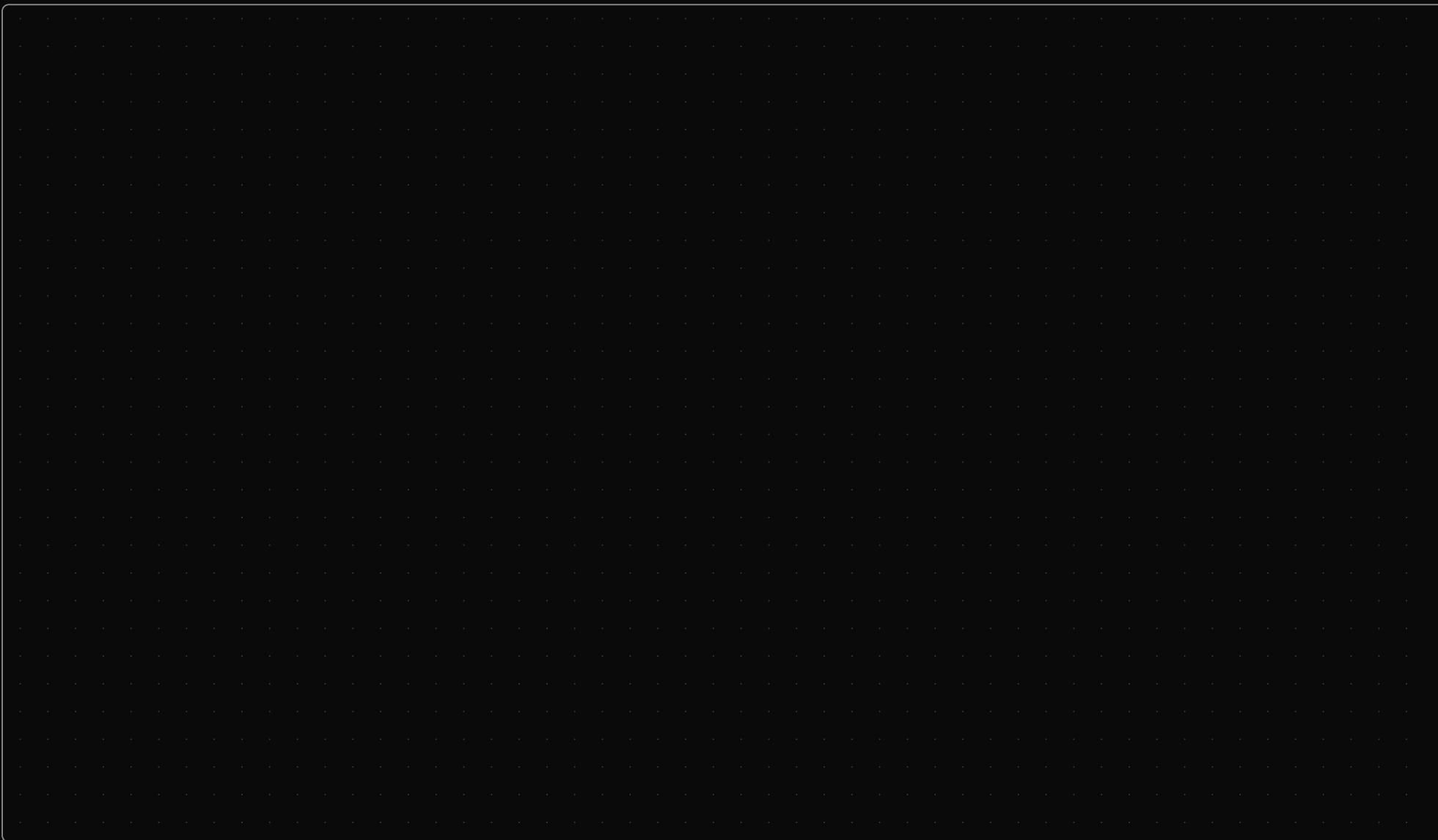
Når bruker du hva?

<code>def</code>	flere steg med logikk, eller skal brukes mange steder
<code>lambda</code>	én enkel regneoperasjon der og da — en «bruk og kast»-funksjon

Du trenger ikke `lambda` i dag. Men når vi kommer til Pandas og skal justere en hel kolonne med tall, sparer den mye skriving.

Notater

Plass til dine egne notater



Oppsummering

Kjeden vi gikk gjennom

1. Kode som gjentas $\Rightarrow$ lag en funksjon
2. Funksjonen er en fabrikk: $\text{inn} \rightarrow \text{maskineri} \rightarrow \text{ut}$
3. `def`, navn, parametere, kolon, innrykk, `return`
4. Argumentene fyller parameterne — i *rekkefølge*
5. `return` sender verdien tilbake; `print` bare viser den

De tre tingene å huske

1. Repeterer du kode, mangler du en funksjon.
2. `print` snakker til **mennesket**. `return` snakker til **programmet**. Uten `return` får du `None`.
3. Python sier ikke fra når du bytter om argumentene. Bare du kan se det.

Veien videre (kapittel 4): lister, tupler, dictionaries og NumPy. Da får du tusenvis av tall å regne på samtidig — og det er funksjonene fra i dag som gjør at koden ikke blir uleselig.

Notater

Til neste gang

Kod selv

1. Lag en funksjon som returnerer hvor mye du har igjen av et budsjett b når du har brukt x .
2. Lag `parking_cost(time_period, p)` der tiden er en streng på formen "`hh:mm:ss`".
3. Lag en funksjon som returnerer nåverdien av et beløp, $X/(1+r)^T$ — og én med kontinuerlig forrentning.

Tenk gjennom

1. Hvorfor gir `len('156')` svaret 3 og ikke 156?
2. En funksjon uten `return` «virker» i notatboka. Når merker du at den likevel er ødelagt?
3. Hvorfor er det lurt at `round()` ligger *inne* i funksjonen og ikke utenfor?

Notatboka: 3 – Funksjoner.ipynb, oppgave 1–6. • Kompendiet: kapittel 3 og Kapittelcase 3.